Project Design Phase-II

Solution Requirements (Functional & Non-functional)

Date	6 July 2026
Team ID	XXXXXX
Project Name	Credit Card Fraud Detection System using Machine Learning
Maximum Marks	4 Marks

FUNCTIONAL REQUIREMENTS

Following are the functional requirements of the proposed solution.

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	Application Navigation	User can open the home page, view project information, navigate to the prediction page, view real-time results, and access documentation/readme details.
FR-2	Transaction Detail Entry	User can enter structural transaction data parameters: Time, Amount, and PCA features V1 through V28.
FR-3	Input Validation	System validates required form input fields, checks numeric parameters, ensures structural bounds, and prevents malformed request payloads before triggering prediction.
FR-4	Prediction Request	HTML/CSS frontend form captures input arrays and submits data asynchronously to the Flask backend controller API route handler.
FR-5	Fraud Pattern Inference	Flask backend uses Joblib to load the trained machine learning pipeline parameters (XGBoost/Random Forest) and computes real-time fraud classifications.
FR-6	Result Display	User can view clear system classification output banners explicitly declaring either "Normal Transaction" or "Fraud Transaction Detected."
FR-7	Backend Health Check	System exposes an active endpoint to check application and model availability metrics on the hosting environment.
FR-8	Cloud Deployment	The integrated codebase repository is managed via GitHub version control and dynamically deployed into production on the Render cloud platform.

NON-FUNCTIONAL REQUIREMENTS

Following are the non-functional requirements of the proposed solution.

NFR No.	Non-Functional Requirement	Description
NFR-1	Usability	Clean HTML/CSS layout providing structured form interfaces for simple input formatting of transaction attributes.
NFR-2	Security	Backend routes implement input validation and error parsing boundaries to protect underlying Python pipelines from unhandled data exceptions.
NFR-2	Reliability	Handles empty features, null inputs, backend exceptions, or missing deployment weights gracefully without dropping active server cycles.
NFR-4	Performance	Machine learning models are loaded optimally once during bootstrap cycles using Joblib, ensuring minimal inference latency through standard JSON pathways.
NFR-5	Availability	Code base deployed on Render cloud platform architecture to maintain persistent accessible endpoints across public web URLs.
NFR-6	Maintainability	Modular design decoupling the data preprocessing engine, machine learning model pipeline scripts, and Flask controller routes for streamlined modifications.